

Assignment-12.4

Name: Sushma Purella

H.No:2303A52188

Batch:44

Task 1: Bubble Sort for Ranking Exam Scores

You are working on a college result processing system where a small list of student scores needs to be sorted after every internal assessment

Prompt:

Implement Bubble Sort in Python to sort a list of student exam scores. Include inline comments explaining comparisons, swaps, and iteration passes.

Also implement an optimized version with early termination and analyze time complexity.

Code:

```
Assignment-12.4.py > ...
1 #Implement Bubble Sort in Python to sort a list of student scores.
2 #Insert inline comments explaining key operations such as comparisons, swaps, and iteration passes
3 def bubble_sort(student_scores):
4     n = len(student_scores)
5     # Outer loop for number of passes
6     for i in range(n):
7         # Inner loop for comparisons in each pass
8         for j in range(0, n - i - 1):
9             # Compare adjacent elements
10            if student_scores[j] > student_scores[j + 1]:
11                # Swap if they are in wrong order
12                student_scores[j], student_scores[j + 1] = student_scores[j + 1], student_scores[j]
13        return student_scores
14 # Example usage
15 scores = [85, 92, 78, 90, 88]
16 sorted_scores = bubble_sort(scores)
17 print("Sorted student scores:", sorted_scores)
18
19 #Identify early-termination conditions when the list becomes sorted
20 def bubble_sort_optimized(student_scores):
21     n = len(student_scores)
22     for i in range(n):
23         swapped = False # Flag to track if any swaps occurred in this pass
24         for j in range(0, n - i - 1):
25             if student_scores[j] > student_scores[j + 1]:
26                 student_scores[j], student_scores[j + 1] = student_scores[j + 1], student_scores[j]
27                 swapped = True # Mark that a swap occurred
28         if not swapped: # If no swaps occurred, the list is already sorted
29             break
30     return student_scores
31
32 #Provide a brief time complexity analysis
33 # The time complexity of Bubble Sort is O(n^2) in the worst and average cases, as it requires two nested loops to compare and swap elements
34 # In the best case, when the list is already sorted, the time complexity is O(n) due to the early termination condition that checks for swaps
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

```
PS C:\Users\sushm\OneDrive\Attachments\Desktop\Ai-Assist> & C:/Users/sushm/AppData/Local/Programs/Python/Python313/python.exe c:/Users/sushm/OneDrive/Attachments/Desktop/Ai-Assist/Assignment-12.4.py
Sorted student scores: [78, 85, 88, 90, 92]
```

```
Assignment-12.4.py > ...
16 sorted_scores = bubble_sort(scores)
17 print("Sorted student scores:", sorted_scores)
18
19 #Identify early-termination conditions when the list becomes sorted
20 def bubble_sort_optimized(student_scores):
21     n = len(student_scores)
22     for i in range(n):
23         swapped = False # Flag to track if any swaps occurred in this pass
24         for j in range(0, n - i - 1):
25             if student_scores[j] > student_scores[j + 1]:
26                 student_scores[j], student_scores[j + 1] = student_scores[j + 1], student_scores[j]
27                 swapped = True # Mark that a swap occurred
28         if not swapped: # If no swaps occurred, the list is already sorted
29             break
30     return student_scores
31
32 #Provide a brief time complexity analysis
33 # The time complexity of Bubble Sort is O(n^2) in the worst and average cases, as it requires two nested loops to compare and swap elements
34 # In the best case, when the list is already sorted, the time complexity is O(n) due to the early termination condition that checks for swaps
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

Observation:

Bubble Sort repeatedly compares adjacent elements and swaps them if they are in the wrong order.

After each pass, the largest unsorted element moves to its correct position.

In the optimized version, if no swaps occur during a pass, the algorithm stops early (early termination).

- Time Complexity:
- Best Case: $O(n)$ (already sorted list with optimization)

- Average Case: $O(n^2)$
- Worst Case: $O(n^2)$

Task 2: Improving Sorting for Nearly Sorted Attendance Records

You are maintaining an attendance system where student roll numbers are already almost sorted, with only a few late updates.

Prompt:

Implement Bubble Sort and Insertion Sort in Python to sort student roll numbers in an attendance system where the data is nearly sorted. Compare their performance on nearly sorted input and explain why one performs better.

Code:

```

36 #attendance system where student roll numbers are already almost sorted, with only a few late updates.
37 #Start with a Bubble Sort Implementation.
38 def bubble_sort_attendance(roll_numbers):
39     n = len(roll_numbers)
40     for i in range(n):
41         swapped = False # Flag to track if any swaps occurred in this pass
42         for j in range(0, n - i - 1):
43             if roll_numbers[j] > roll_numbers[j + 1]:
44                 roll_numbers[j], roll_numbers[j + 1] = roll_numbers[j + 1], roll_numbers[j]
45                 swapped = True # Mark that a swap occurred
46         if not swapped: # If no swaps occurred, the list is already sorted
47             break
48     return roll_numbers
49
50 # Example usage
51 roll_numbers = [101, 102, 103, 105, 104, 106]
52 sorted_roll_numbers = bubble_sort_attendance(roll_numbers)
53 print("Sorted roll numbers:", sorted_roll_numbers)
54
55 #Generate an Insertion Sort implementation
56 def insertion_sort_attendance(roll_numbers):
57     n = len(roll_numbers)
58     for i in range(1, n):
59         key = roll_numbers[i] # The current element to be positioned
60         j = i - 1 # The index of the last sorted element
61         # Move elements of roll_numbers[0..i-1], that are greater than key,
62         # to one position ahead of their current position
63         while j >= 0 and roll_numbers[j] > key:
64             roll_numbers[j + 1] = roll_numbers[j] # Shift the element to the right
65             j -= 1
66         roll_numbers[j + 1] = key
67
68 PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Sorted roll numbers: [101, 102, 103, 104, 105, 106]
Sorted roll numbers using Insertion Sort: [101, 102, 103, 104, 105, 106]
Bubble Sort execution time on nearly sorted data: 1.2159347534179688e-05 seconds
Insertion Sort execution time on nearly sorted data: 2.193450927734375e-05 seconds

```

```
85     j -= 1
86     roll_numbers[j + 1] = key # Place the key in its correct position
87     return roll_numbers
88 # Example usage
89 roll_numbers = [101, 102, 103, 105, 104, 106]
90 sorted_roll_numbers = insertion_sort_attendance(roll_numbers)
91 print("Sorted roll numbers using Insertion Sort:", sorted_roll_numbers)
92
93 # Explain why Insertion Sort performs better on nearly sorted data
94 # Insertion Sort performs better on nearly sorted data because it has a time complexity of O(n) in the best case scenario,
95 # where the data is already sorted or nearly sorted. In such cases, each element is only compared with a few
96 # elements before being inserted into its correct position, making the process very efficient.
97
98 # Compare execution behavior on nearly sorted input
99 import time
100 # Generate a nearly sorted list of roll numbers
101 nearly_sorted_roll_numbers = [101, 102, 103, 105, 104, 106] # Only one element is out of order
102 # Measure execution time for Bubble Sort
103 start_time = time.time()
104 bubble_sort_attendance(nearly_sorted_roll_numbers.copy()) # Use copy to avoid modifying the original list
105 end_time = time.time()
106 print("Bubble Sort execution time on nearly sorted data:", end_time - start_time, "seconds")
107 # Measure execution time for Insertion Sort
108 start_time = time.time()
109 insertion_sort_attendance(nearly_sorted_roll_numbers.copy()) # Use copy to avoid modifying the original list
110 end_time = time.time()
111 print("Insertion Sort execution time on nearly sorted data:", end_time - start_time, "seconds")
112
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

orted roll numbers: [101, 102, 103, 104, 105, 106]
orted roll numbers using Insertion Sort: [101, 102, 103, 104, 105, 106]
ubble Sort execution time on nearly sorted data: 1.2159347534179688e-05 seconds
nsertion Sort execution time on nearly sorted data: 2.193450927734375e-05 seconds

Observation:

Bubble Sort compares adjacent elements and swaps them repeatedly. Even with optimization, it may still require multiple passes.

Insertion Sort inserts each element into its correct position within the already sorted portion of the list.

For nearly sorted data, Insertion Sort performs fewer shifts and comparisons.

Time Complexity:

Bubble Sort:

Best Case: $O(n)$ (with early termination)

Average/Worst Case: $O(n^2)$

Insertion Sort:

Best Case: $O(n)$ (already/nearly sorted)

Average/Worst Case: $O(n^2)$

In practice, Insertion Sort executes faster than Bubble Sort for nearly sorted roll numbers because it minimizes unnecessary swaps.

Task 3: Searching Student Records in a Database

You are developing a student information portal where users search for student records by roll number.

Prompt:

Implement Linear Search and Binary Search in Python to search student records by roll number.

Add proper docstrings explaining parameters and return values.

Compare both searching techniques and explain when Binary Search is applicable.

Code:

```
Assignment-12.4.py > ...
96 #student information portal where users search for student records by roll number.
97 #Linear Search for unsorted student data
98 ##Add docstrings explaining parameters and return values
99 def linear_search(student_records, roll_number):
100     """
101     Performs a linear search on a list of student records to find a record by roll number.
102
103     Parameters:
104         student_records (list): A list of dictionaries representing student records.
105         roll_number (int): The roll number to search for.
106
107     Returns:
108         dict or None: The matching student record if found, otherwise None.
109     """
110     for record in student_records:
111         if record['roll_number'] == roll_number: # Compare the roll number with the current record
112             return record # Return the matching record if found
113     return None # Return None if no matching record is found
114
115 # Example usage
116 student_records = [
117     {'roll_number': 101, 'name': 'Alice'},
118     {'roll_number': 102, 'name': 'Bob'},
119     {'roll_number': 103, 'name': 'Charlie'}]
120 roll_number_to_search = 102
121 result = linear_search(student_records, roll_number_to_search)
122 if result:
123     print("Student record found:", result)
124 else:
125     print("Student record not found.")
126
127 #Binary Search for sorted student data
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

```
PS C:\Users\sushm\OneDrive\Attachments\Desktop\Ai-Assist> & C:/Users/sushm/AppData/Local/Programs/Python/Python313/python.exe c:/Users/sushm/OneDrive/Attachm
ssignment-12.4.py
Student record found: {'roll_number': 102, 'name': 'Bob'}
Student record found: {'roll_number': 102, 'name': 'Bob'}
```

```
25 #Binary Search for sorted student data
26
27 def binary_search(student_records, roll_number):
28     """
29     Performs a binary search on a sorted list of student records to find a record by roll number.
30
31     Parameters:
32     | student_records (list): A sorted list of dictionaries representing student records.
33     | roll_number (int): The roll number to search for.
34
35     Returns:
36     | dict or None: The matching student record if found, otherwise None.
37     """
38     left, right = 0, len(student_records) - 1
39     while left <= right:
40         mid = left + (right - left) // 2 # Calculate the middle index
41         if student_records[mid]['roll_number'] == roll_number: # Check if the middle record matches the roll number
42             return student_records[mid] # Return the matching record if found
43         elif student_records[mid]['roll_number'] < roll_number: # If the middle record's roll number is less than the target
44             left = mid + 1 # Search in the right half
45         else: # If the middle record's roll number is greater than the target
46             right = mid - 1 # Search in the left half
47     return None # Return None if no matching record is found
48
49 # Example usage
50 sorted_student_records = sorted(student_records, key=lambda x: x['roll_number']) #
51 roll_number_to_search = 102
52 result = binary_search(sorted_student_records, roll_number_to_search)
53 if result:
54     print("Student record found:", result)
55 else:
56     print("Student record not found.")
57
58 # Explain when Binary Search is applicable
59 # Binary Search is applicable when the data is sorted. It works by repeatedly dividing the search interval in half,
60 # which allows it to find the target value efficiently. If the data is not sorted, Binary Search cannot be used,
61 # and a different search algorithm, such as Linear Search, would be necessary.
62
63 # Differences between the two searches
64 # Linear Search:
65 # - Time Complexity: O(n) in worst case, O(1) in best case.
66 # - Space Complexity: O(1).
67 # - Works on both sorted and unsorted data.
68 # - Simple to implement and understand.
69
70 # Binary Search:
71 # - Time Complexity: O(log n).
72 # - Space Complexity: O(1).
73 # - Works only on sorted data.
74 # - More efficient for large datasets but requires data to be sorted.
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\sushm\OneDrive\Attachments\Desktop\AI-Assist> & C:/Users/sushm/AppData/Local/Programs/Python/Python313/python.exe c:/Users/sushm/OneDrive/Attachments/Desktop/AI-Assist/Assignment-12.4.py

Student record found: {'roll_number': 102, 'name': 'Bob'}

Student record found: {'roll_number': 102, 'name': 'Bob'}

```
154 else:
155     print("Student record not found.")
156
157 # Explain when Binary Search is applicable
158 # Binary Search is applicable when the data is sorted. It works by repeatedly dividing the search interval in half,
159 # which allows it to find the target value efficiently. If the data is not sorted, Binary Search cannot be used,
160 # and a different search algorithm, such as Linear Search, would be necessary.
161
162
163 # Differences between the two searches
164 # Linear Search:
165 # - Time Complexity: O(n) in worst case, O(1) in best case.
166 # - Space Complexity: O(1).
167 # - Works on both sorted and unsorted data.
168 # - Simple to implement and understand.
169
170 # Binary Search:
171 # - Time Complexity: O(log n).
172 # - Space Complexity: O(1).
173 # - Works only on sorted data.
174 # - More efficient for large datasets but requires data to be sorted.
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\sushm\OneDrive\Attachments\Desktop\AI-Assist> & C:/Users/sushm/AppData/Local/Programs/Python/Python313/python.exe c:/Users/sushm/OneDrive/Attachments/Desktop/AI-Assist/Assignment-12.4.py

Student record found: {'roll_number': 102, 'name': 'Bob'}

Student record found: {'roll_number': 102, 'name': 'Bob'}

Observation:

- Linear Search checks each student record one by one until the roll number is found.
- It works on both sorted and unsorted data.
- Time Complexity of Linear Search:
 - Best Case: $O(1)$

- Worst Case: $O(n)$
- Binary Search repeatedly divides the sorted data into halves to find the roll number.
- It works only when the student records are sorted.
- Time Complexity of Binary Search:
 - Best/Worst Case: $O(\log n)$
- For small or unsorted student data, Linear Search is suitable.
- For large and sorted datasets, Binary Search is much more efficient.

Task 4: Choosing Between Quick Sort and Merge Sort for Data Processing

You are part of a data analytics team that needs to sort large datasets received from different sources (random order, already sorted, and reverse sorted).

Prompt:

Implement recursive Quick Sort and Merge Sort algorithms in Python to sort large datasets.

Add meaningful docstrings explaining parameters and return values.

Explain how recursion works in each algorithm.

Test both algorithms on random, already sorted, and reverse-sorted data.

Code:

```

187 def quick_sort(arr):
188     """
189     Sorts an array using the Quick Sort algorithm.
190
191     Parameters:
192     arr (list): The list of elements to be sorted.
193
194     Returns:
195     list: A new sorted list.
196     """
197     if len(arr) <= 1: # Base case: arrays with 0 or 1 element are already sorted
198         return arr
199     pivot = arr[len(arr) // 2] # Choose the middle element as the pivot
200     left = [x for x in arr if x < pivot] # Elements less than the pivot
201     middle = [x for x in arr if x == pivot] # Elements equal to the pivot
202     right = [x for x in arr if x > pivot] # Elements greater than the pivot
203     return quick_sort(left) + middle + quick_sort(right) # Recursively sort left and right partitions
204 def merge_sort(arr):
205     """Sorts an array using the Merge Sort algorithm.
206
207     Parameters:
208     arr (list): The list of elements to be sorted.
209     Returns:
210     list: A new sorted list.
211     """
212     if len(arr) <= 1: # Base case: arrays with 0 or 1 element are already sorted

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\sushm\OneDrive\Attachments\Desktop\Ai-Assist> & C:/Users/sushm/AppData/Local/Programs/Python/Python313/python.exe c:/Users/sushm/OneDrive/Attachments/Desktop/ssignment-12.4.py

Quick Sort on sorted data: [1, 2, 3, 4, 5, 6]
 Quick Sort on reverse-sorted data: [1, 2, 3, 4, 5, 6]
 Merge Sort on random data: [1, 2, 5, 5, 6, 9]
 Merge Sort on sorted data: [1, 2, 3, 4, 5, 6]
 Merge Sort on reverse-sorted data: [1, 2, 3, 4, 5, 6]

```

212     if len(arr) <= 1: # Base case: arrays with 0 or 1 element are already sorted
213         return arr
214     mid = len(arr) // 2 # Find the middle index
215     left_half = merge_sort(arr[:mid]) # Recursively sort the left half
216     right_half = merge_sort(arr[mid:]) # Recursively sort the right half
217     return merge(left_half, right_half) # Merge the sorted halves
218 def merge(left, right):
219     """Merges two sorted lists into a single sorted list.
220
221     Parameters:
222     left (list): The first sorted list.
223     right (list): The second sorted list.
224
225     Returns:
226     list: A merged and sorted list containing all elements from both input lists.
227     """
228     result = []
229     i = j = 0
230     while i < len(left) and j < len(right):
231         if left[i] < right[j]: # Compare elements from both lists
232             result.append(left[i]) # Add the smaller element to the result
233             i += 1 # Move the pointer in the left list
234         else:
235             result.append(right[j]) # Add the smaller element to the result
236             j += 1 # Move the pointer in the right list

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\sushm\OneDrive\Attachments\Desktop\Ai-Assist> & C:/Users/sushm/AppData/Local/Programs/Python/Python313/python.exe c:/Users/sushm/OneDrive/Attachments/Desktop/Ai-Assist/A

ssignment-12.4.py
 Quick Sort on sorted data: [1, 2, 3, 4, 5, 6]
 Quick Sort on reverse-sorted data: [1, 2, 3, 4, 5, 6]
 Merge Sort on random data: [1, 2, 5, 5, 6, 9]
 Merge Sort on sorted data: [1, 2, 3, 4, 5, 6]
 Merge Sort on reverse-sorted data: [1, 2, 3, 4, 5, 6]


```
234         else:
235             result.append(right[j]) # Add the smaller element to the result
236             j += 1 # Move the pointer in the right list
237         result.extend(left[i:]) # Add any remaining elements from the left list
238         result.extend(right[j:]) # Add any remaining elements from the right list
239         return result
240
241     # Test both algorithms on random data, sorted data, and reverse-sorted data
242     random_data = [5, 2, 9, 1, 5, 6]
243     sorted_data = [1, 2, 3, 4, 5, 6]
244     reverse_sorted_data = [6, 5, 4, 3, 2, 1]
245     print("Quick Sort on random data:", quick_sort(random_data))
246     print("Quick Sort on sorted data:", quick_sort(sorted_data))
247     print("Quick Sort on reverse-sorted data:", quick_sort(reverse_sorted_data))
248     print("Merge Sort on random data:", merge_sort(random_data))
249     print("Merge Sort on sorted data:", merge_sort(sorted_data))
250     print("Merge Sort on reverse-sorted data:", merge_sort(reverse_sorted_data))
251
252
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

```
PS C:\Users\sushm\OneDrive\Attachments\Desktop\Ai-Assist> & C:/Users/sushm/AppData/Local/Programs/Python/Python313/python.exe c:/Users/sushm/OneDrive/Attachments/ssignment-12.4.py
Quick Sort on sorted data: [1, 2, 3, 4, 5, 6]
Quick Sort on reverse-sorted data: [1, 2, 3, 4, 5, 6]
Merge Sort on random data: [1, 2, 5, 5, 6, 9]
Merge Sort on sorted data: [1, 2, 3, 4, 5, 6]
Merge Sort on reverse-sorted data: [1, 2, 3, 4, 5, 6]
```

Observation:

- Quick Sort and Merge Sort both use recursion and follow the Divide and Conquer approach.
- Quick Sort divides the list around a pivot and recursively sorts left and right parts.
- Merge Sort divides the list into halves and recursively merges them in sorted order.
- Merge Sort always runs in $O(n \log n)$ time.
- Quick Sort runs in $O(n \log n)$ on average but $O(n^2)$ in the worst case (bad pivot choice).

Task 5: Optimizing a Duplicate Detection Algorithm

You are building a data validation module that must detect duplicate user IDs in a large dataset before importing it into a system.

Prompt:

Implement a naive duplicate detection algorithm using nested loops to detect duplicate user IDs.

Analyze its time complexity. Then implement an optimized version

using sets and compare their performance conceptually for large datasets.

Code:

```
254
255 #data validation module that must detect duplicate user IDs in a large dataset before importing it into a system.
256 #a naive duplicate detection algorithm using nested loops.
257 def detect_duplicates_naive(user_ids):
258     """
259     Detects duplicate user IDs in a list using a naive approach with nested loops.
260
261     Parameters:
262         user_ids (list): A list of user IDs to check for duplicates.
263
264     Returns:
265         set: A set of duplicate user IDs found in the list.
266     """
267     duplicates = set() # Set to store duplicate user IDs
268     n = len(user_ids)
269     for i in range(n):
270         for j in range(i + 1, n): # Compare each user ID with the rest of the list
271             if user_ids[i] == user_ids[j]: # Check for a duplicate
272                 duplicates.add(user_ids[i]) # Add the duplicate to the set
273     return duplicates
274
275 #Analyze the time complexity
276 # The time complexity of the naive duplicate detection algorithm is  $O(n^2)$  in the worst case,
277 # because it uses nested loops to compare each user ID with every other user ID in the list.
278
279 #Suggest an optimized approach using sets or dictionaries
280 def detect_duplicates_optimized(user_ids):
281     """
282     Detects duplicate user IDs in a list using an optimized approach with a set.
283
284     Parameters:
285
286     Returns:
287         set: A set of duplicate user IDs found in the list.
288     """
289     seen = set() # Set to track seen user IDs
290     duplicates = set() # Set to store duplicate user IDs
291     for user_id in user_ids:
292         if user_id in seen: # Check if the user ID has already been seen
293             duplicates.add(user_id) # Add to duplicates if it's already seen
294         else:
295             seen.add(user_id) # Add to seen if it's not already there
296     return duplicates
297
298 # Example usage
299 user_ids = ['user1', 'user2', 'user3', 'user1', 'user4', 'user2']
300 print("Naive duplicate detection:", detect_duplicates_naive(user_ids))
301 print("Optimized duplicate detection:", detect_duplicates_optimized(user_ids))#
302
303 #Rewrite the algorithm with improved efficiency
304 #Compare execution behavior conceptually for large input sizes
305
306 # The optimized duplicate detection algorithm has a time complexity of  $O(n)$  in the worst case,
307 # because it iterates through the list of user IDs once, while the naive approach has a time complexity of  $O(n^2)$ 
308 # due to the nested loops. For large input sizes, the optimized approach will significantly outperform the naive approach,
309 # as it avoids redundant comparisons and uses a set to track seen user IDs efficiently.
310
311
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\sushm\OneDrive\Attachments\Desktop\Ai-Assist> & C:/Users/sushm/AppData/Local/Programs/Python/Python313/python.exe c:/Users/sushm/OneDrive/Attachments/Desktop/nment-12.4.py
○ Naive duplicate detection: {'user1', 'user2'}
Optimized duplicate detection: {'user1', 'user2'}
```

```
283
284 Parameters:
285     user_ids (list): A list of user IDs to check for duplicates.
286
287 Returns:
288     set: A set of duplicate user IDs found in the list.
289
290 """
291 seen = set() # Set to track seen user IDs
292 duplicates = set() # Set to store duplicate user IDs
293 for user_id in user_ids:
294     if user_id in seen: # Check if the user ID has already been seen
295         duplicates.add(user_id) # Add to duplicates if it's already seen
296     else:
297         seen.add(user_id) # Add to seen if it's not already there
298     return duplicates
299
300 # Example usage
301 user_ids = ['user1', 'user2', 'user3', 'user1', 'user4', 'user2']
302 print("Naive duplicate detection:", detect_duplicates_naive(user_ids))
303 print("Optimized duplicate detection:", detect_duplicates_optimized(user_ids))#
304
305 #Rewrite the algorithm with improved efficiency
306 #Compare execution behavior conceptually for large input sizes
307
308 # The optimized duplicate detection algorithm has a time complexity of  $O(n)$  in the worst case,
309 # because it iterates through the list of user IDs once, while the naive approach has a time complexity of  $O(n^2)$ 
310 # due to the nested loops. For large input sizes, the optimized approach will significantly outperform the naive approach,
311 # as it avoids redundant comparisons and uses a set to track seen user IDs efficiently.
312
313
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\sushm\OneDrive\Attachments\Desktop\Ai-Assist> & C:/Users/sushm/AppData/Local/Programs/Python/Python313/python.exe c:/Users/sushm/OneDrive/Attachments/Desktop/nment-12.4.py
○ Naive duplicate detection: {'user1', 'user2'}
Optimized duplicate detection: {'user1', 'user2'}
```

Observation:

- The naive approach uses nested loops and compares every pair of user IDs.
- Its time complexity is $O(n^2)$, which is inefficient for large datasets.

- The optimized approach uses a set to track seen user IDs.
- Its time complexity is $O(n)$ since it scans the list only once.
- For large input sizes, the optimized method performs significantly faster and avoids unnecessary comparisons.